

Rapport – Prosjekt, Del 1+2+3 – INF161 H25

Del 1

Fremgangsmåte og begrunnelser

Startet prosjektet med å opprette et filhierarki, der blant annet utforsking av data ble gjort i egne notatbøker og kode som brukes i pipelinen ble gjort i egne .py-filer. I del 1 ble kun *notebooks*, *src* og *output* brukt, men senere blir også *models* introdusert.

notebooks - Brukt for å analysere og utforske rådata og model_ready-datasettet, i tillegg til visualisering. Ble også senere brukt for å utforske modelltrening. I notatbøkene kunne jeg utforske data på en enkel og oversiktlig måte, med umiddelbar tilbakemelding og mer leselig output i forhold til .py-filer. Gjorde både statistisk analyse, i form av funksjoner som *head()*, *info()* og *describe()*, og visualisering, via plotly express. De tre første notatbøkene, *01_explore_stations.ipynb*, *02_explore_trips.ipynb* og *03_explore_weather.ipynb*, ble brukt til å utforske filene separat, mens *04_explore_all* ble brukt til å lage et sammensatt datasett, *merged_df*, av all relevant data fra de andre datasettene. Dette gjorde det enkelt å utforske hvordan dataen hang sammen. Senere la jeg til *05_model_learning.ipynb*, som ble brukt til å utforske ulike regresjonsmodeller og finne beste RMSE på valideringsdata.

src - Brukt for å laste inn, preprosessere og smelte sammen data fra de ulike csv-filene og skrive til *model_ready.csv*. Ble også senere brukt for modelltrening/lagring av modell og predikering. Koden skrevet og utforsket i notatbøkene ble overført hit med noen få viktige endringer, som å endre skrivemåten på ukedagene fra navn til tall i preparasjon for senere modelltrening. Lagde funksjoner for innlasting, preprosessering og sammensmelting i *data_utils.py*, med en kjørbar *pipeline.py* som bruker disse funksjonene for å skrive ferdig datasett til *model_ready.csv*. La senere til funksjoner for modelltrening i *train_models.py*, videreutviklet/erstattet *pipeline.py* med *train.py* og printer predikert data i *predict.py*.

output - Brukt til å holde på *model_ready.csv*.

models - Brukt til å holde på den beste regresjonsmodellen i *best_model.pkl* og feature-kolonnene i *feature_cols.pkl*.

Dataegenskaper (struktur, feil, mangler, innsikter)

stations.csv – All analyse utført ble gjort på det som ville tilsvart treningsdata, altså de første 70% av dataen i *stations.csv*, perioden 10.04/24 – 17.02/25. Inneholdt mange unødvendige kolonner. Beholdte kun kolonnene *timestamp*, *station* og *free_bikes*. Data var riktig formatert ihht *README.md* i *raw_data*. Målingene tok sted over en tilstrekkelig tidsperiode for god analyse, manglet ikke noen verdier og hadde heller ingen nevneverdige uteliggende verdier. Det viktigste å merke seg med dette datasettet er at antall sykler på alle de aktuelle stasjonene var uendret/statiske i en periode fra omtrent 15. april 2024 til 15. august 2024. Endte opp med å omformaterer til et nytt datasett, *target_stations_df*, der de tidligere nevnte radene ble beholdt og formatert ihht LOCF (Last Observation Carried Forward), mens de andre radene ble droppet. Dette gjorde det enklere å visualisere datasettet (se visualiseringer under).

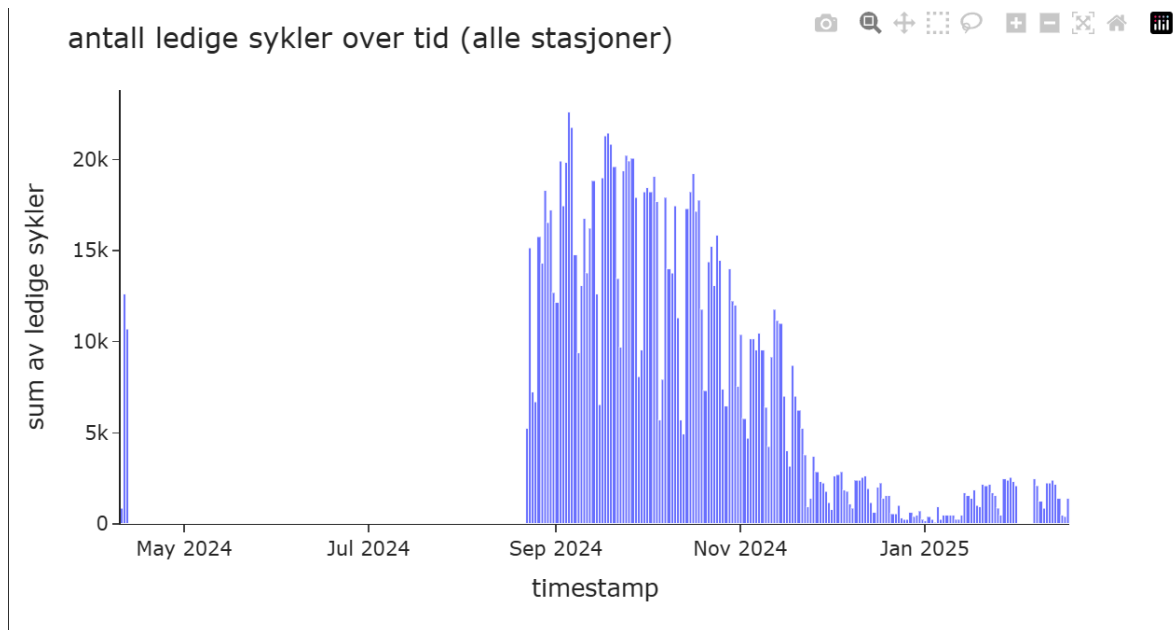
trips.csv – Dataen i *trips.csv* tok sted over lengre tid enn *stations.csv*, så sørget for å kun analysere perioden av *trips.csv* som ville tilsvart perioden for treningsdata (første 70%) i *stations.csv*. Inneholdt noe unødvendig informasjon, men beholdte alt for analysen siden *trips.csv* kun skulle supplere data og ikke være basen for *model_ready.csv*. Dataen var riktig formatert ihht *README* i *raw_data*. Målingene tok sted over en tilstrekkelig tidsperiode for god analyse, manglet ikke noen verdier, antall start- og slutturer var konsistent og dataen hadde ingen nevneverdige uteliggende verdier.

weather.csv – Dataen i *weather.csv* tok også sted over lengre tid enn *stations.csv*, så tok i bruk samme metode som nevnt over for å kun analysere det som ville tilsvart perioden for treningsdata i *stations.csv*. Inneholdt stort sett relevant informasjon. Dataen var riktig formatert ihht *README* i *raw_data*. Målingene tok sted over en tilstrekkelig tidsperiode for god analyse og hadde ingen nevneverdige uteliggende verdier. Datasettet hadde en del manglende verdier som senere ble håndtert via imputasjon.

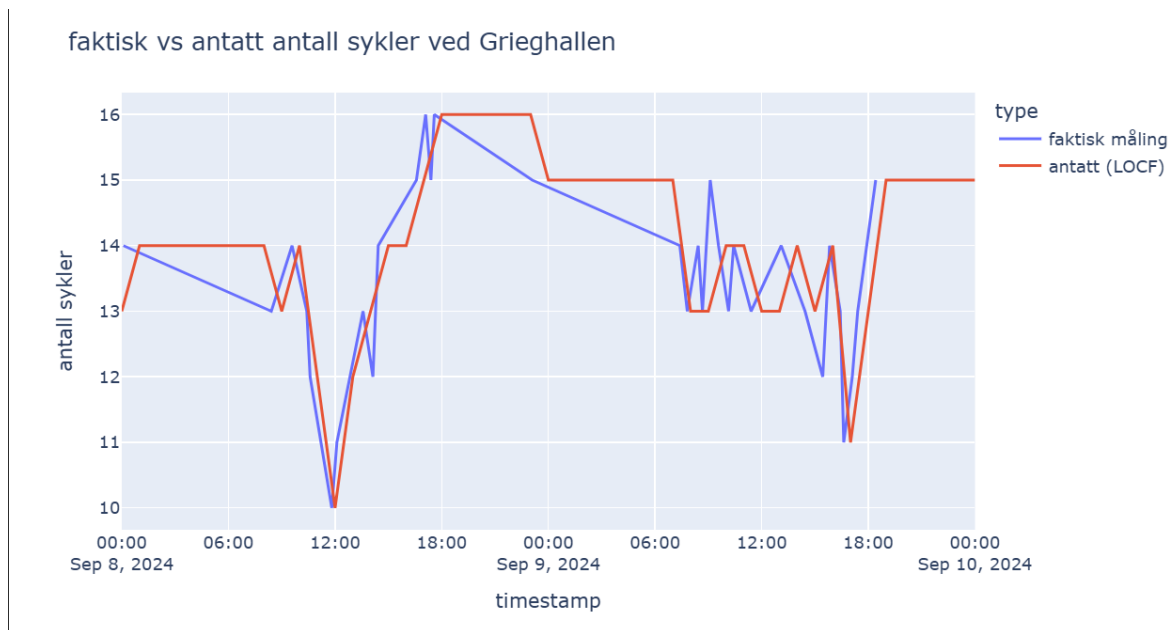
Relevante visualiseringer

stations.csv:

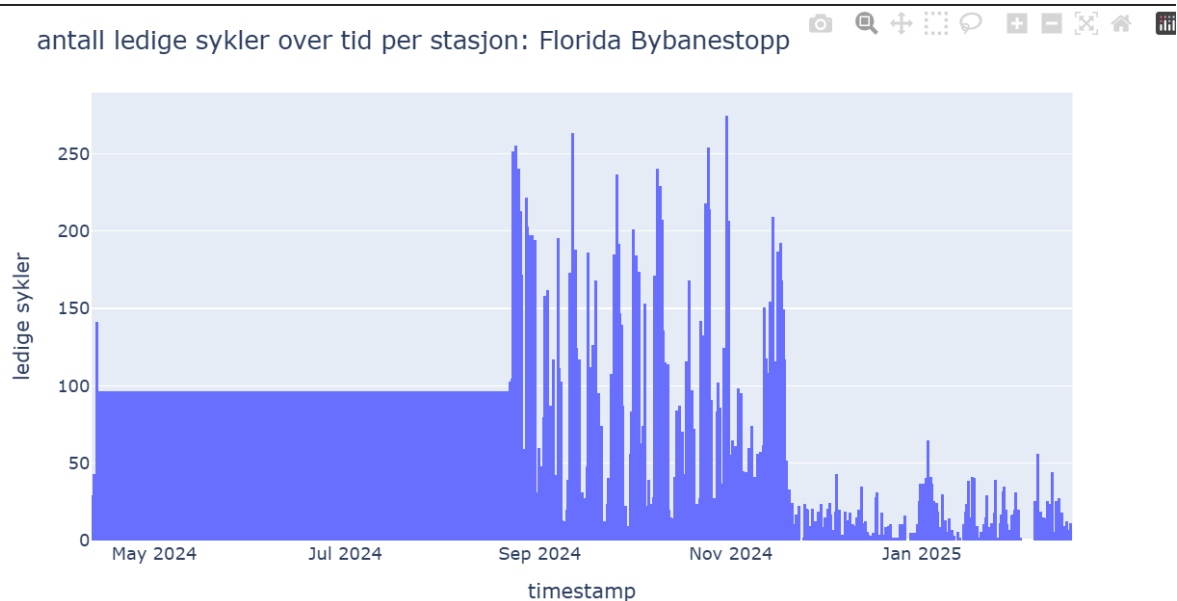
Figur 1 – Observerer sesongpåvirkning på antall ledige sykler. Legg merke til statisk periode fra ~ 15. april til 15. august



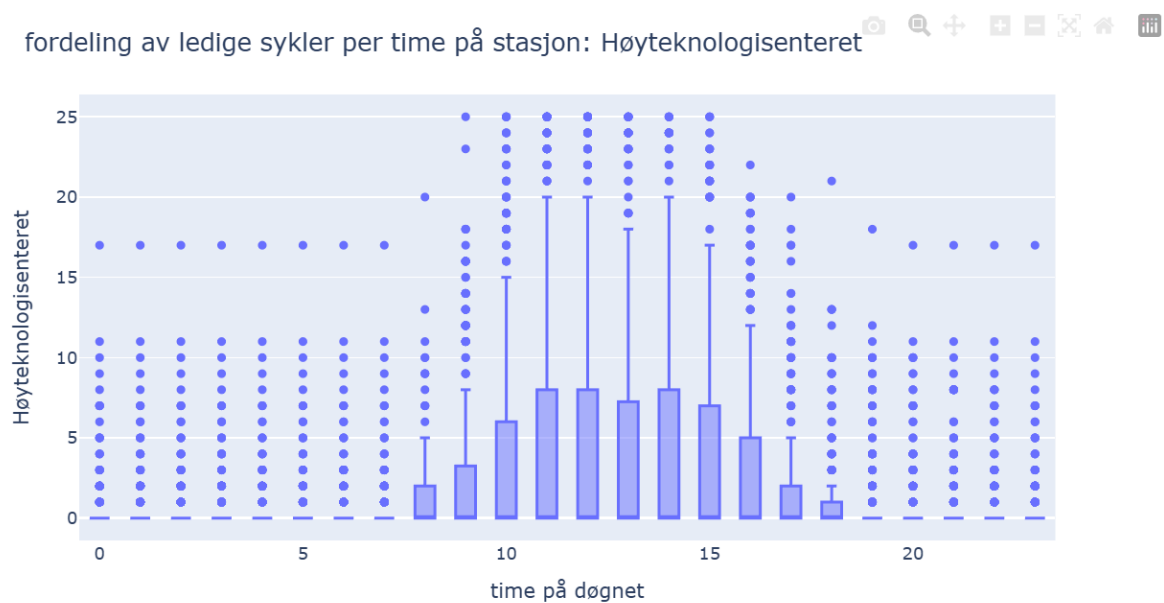
Figur 2 - Sørger for at LOCF-prinsippet ble fulgt riktig i resampling

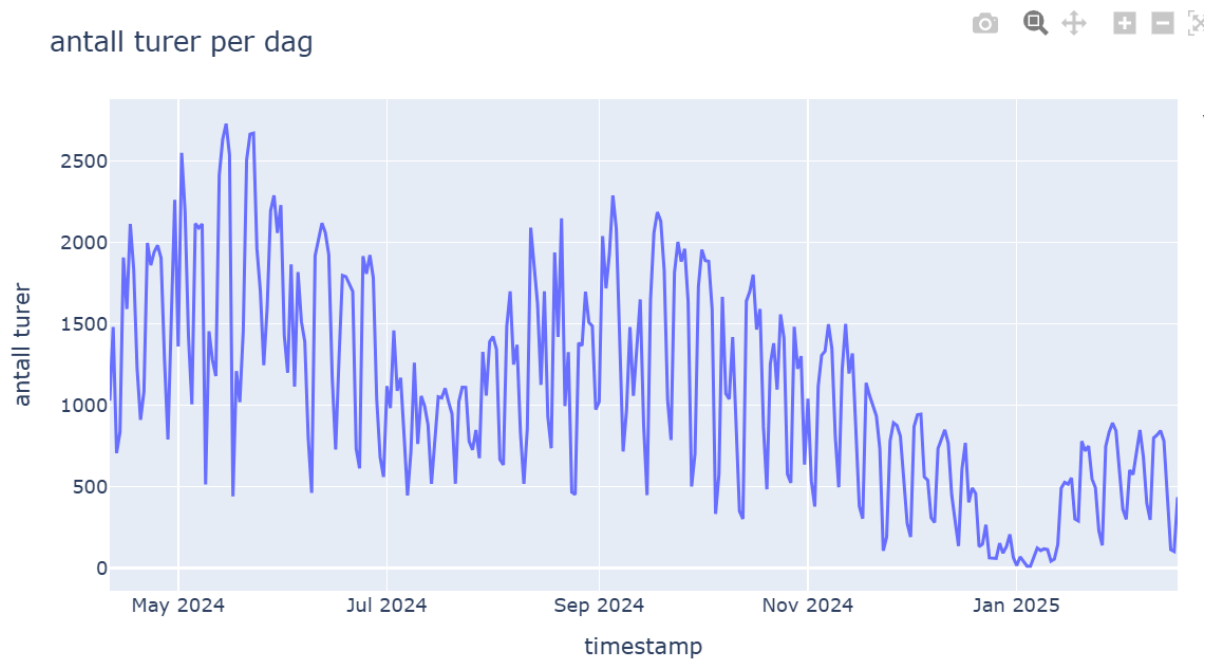
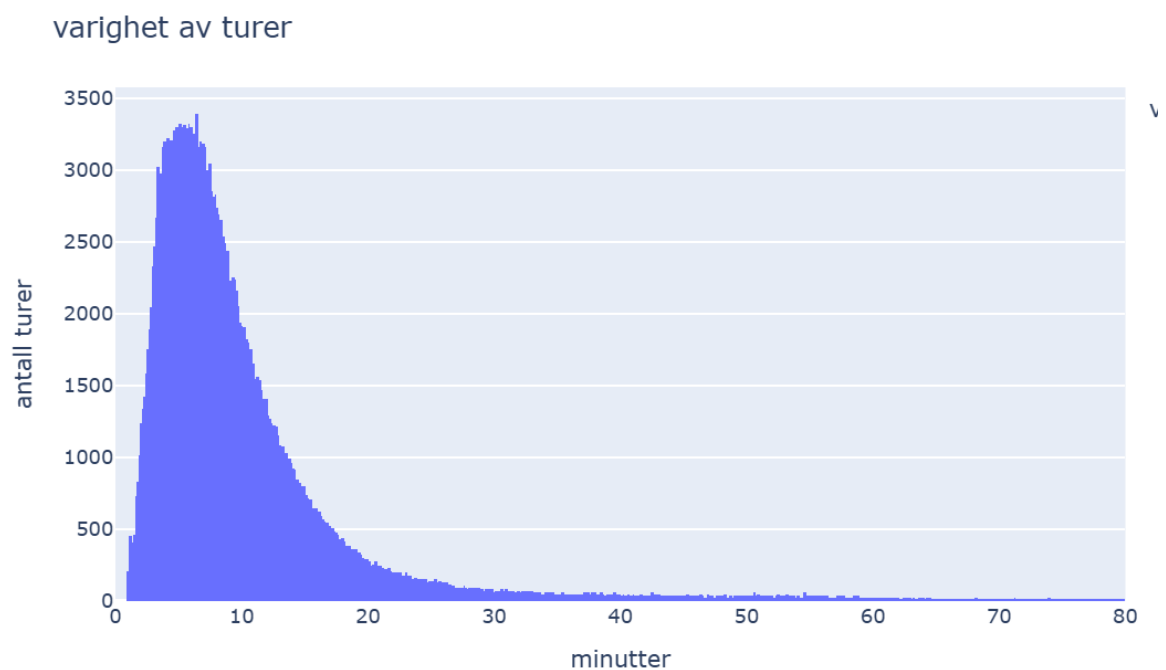


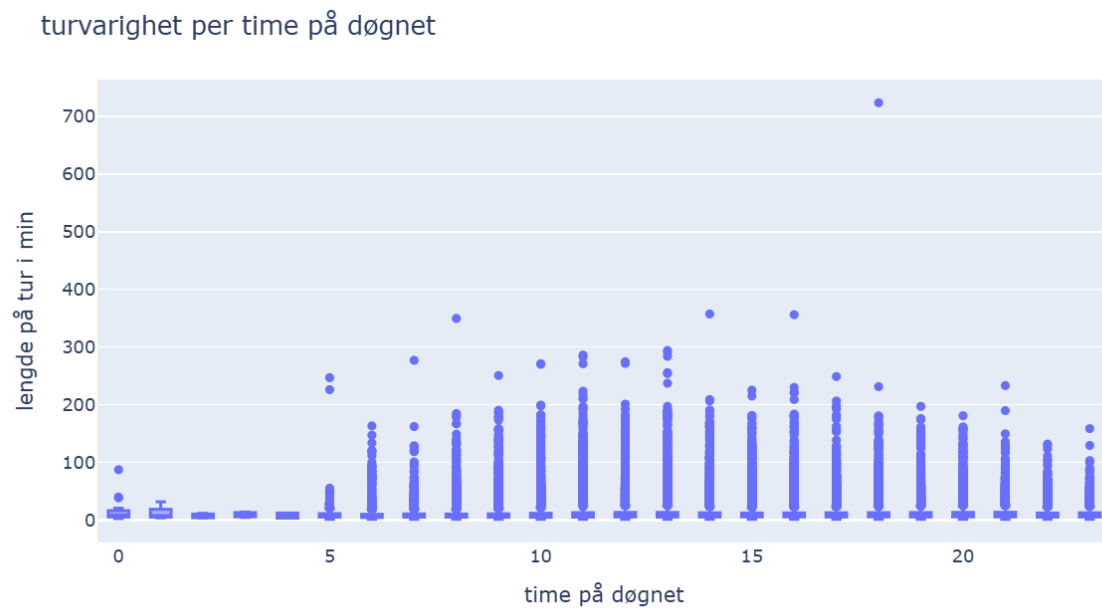
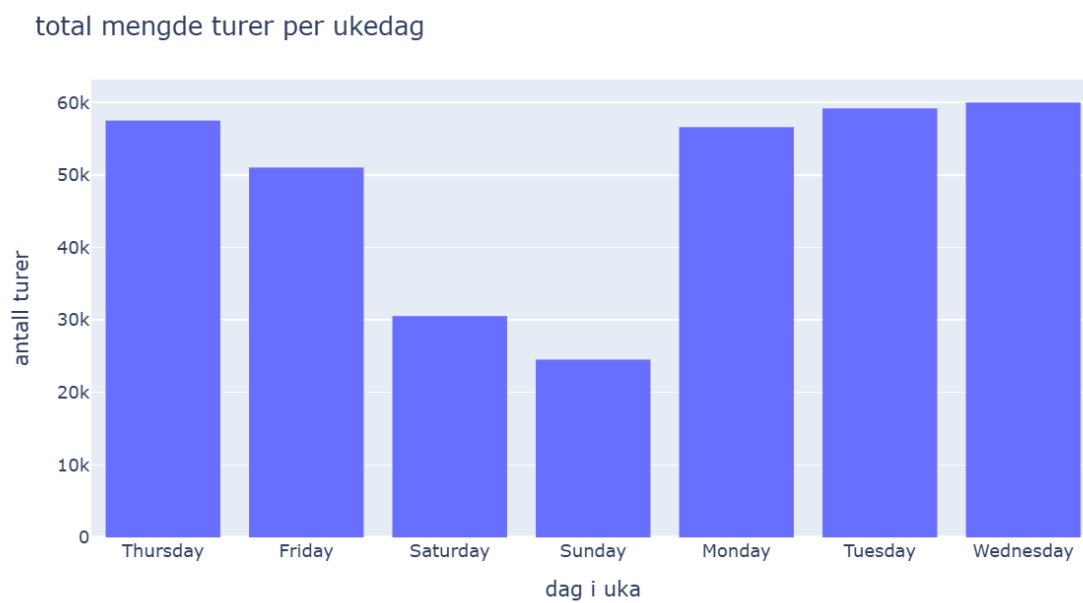
Figur 3 – Antall ledige sykler på tilfeldig valgt stasjon for videre sjekk av sesongmønstre



Figur 4 - Sjekk for uteliggende verdier ved å se på fordeling av ledige sykler i løpet av et døgn på tilfeldig valgt stasjon

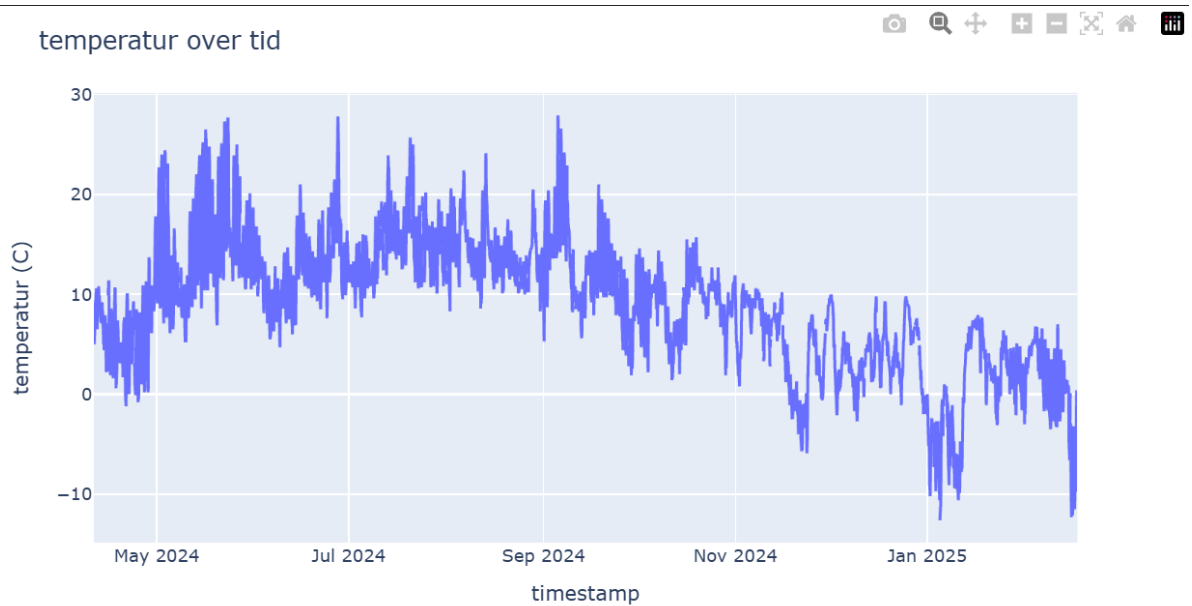


trips.csv:*Figur 5 – Ser på antall sykkelturner per dag for sjekke sesongmønstre**Figur 6 - Ser på turvarighet for å finne gjennomsnittlig turlengde*

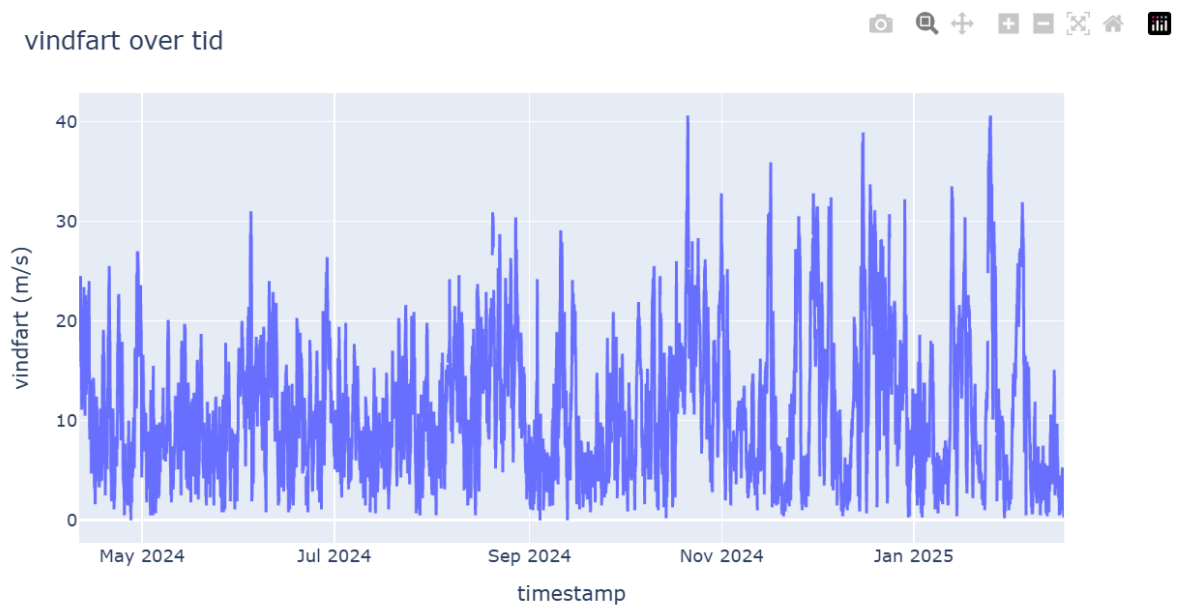
Figur 7 - Ser på turvarighet gjennom døgnet for å finne mest populære tidspunkt å sykle på*Figur 8 - Ser på total mengde turer i uka for å finne mest populære ukedag å sykle på*

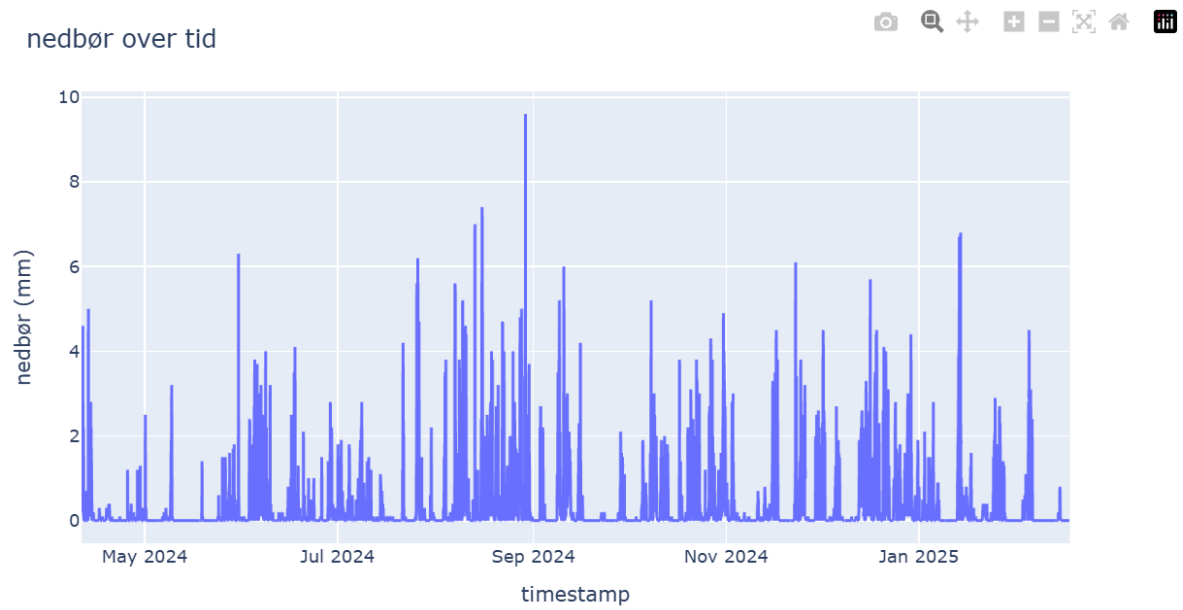
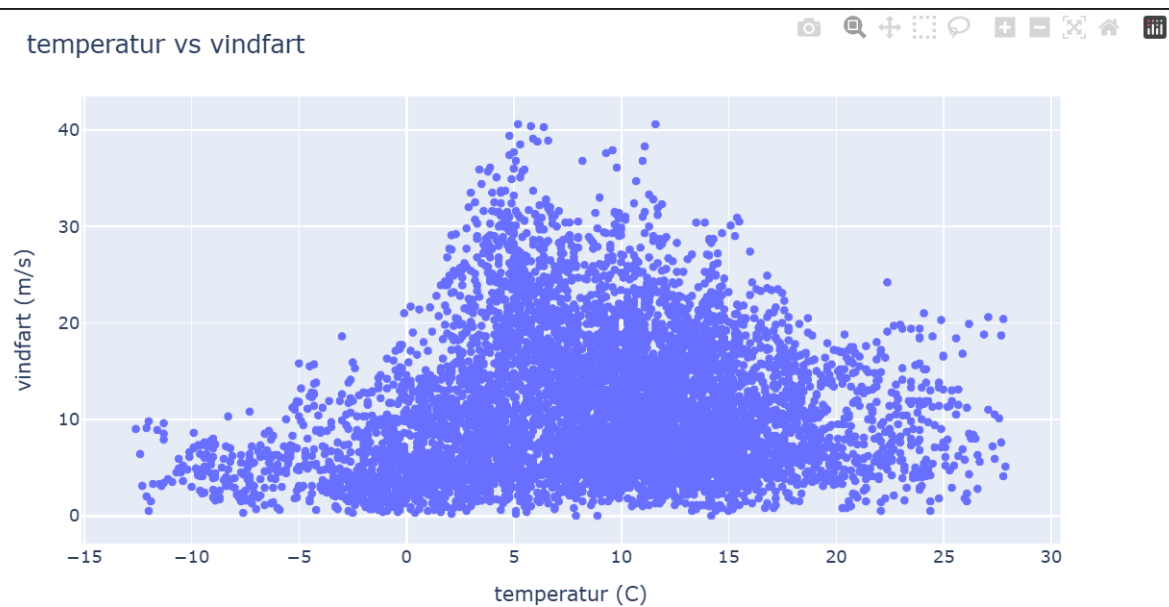
weather.csv:

Figur 9 - Observerer sesongmønstre ved å se på temperatur i perioden



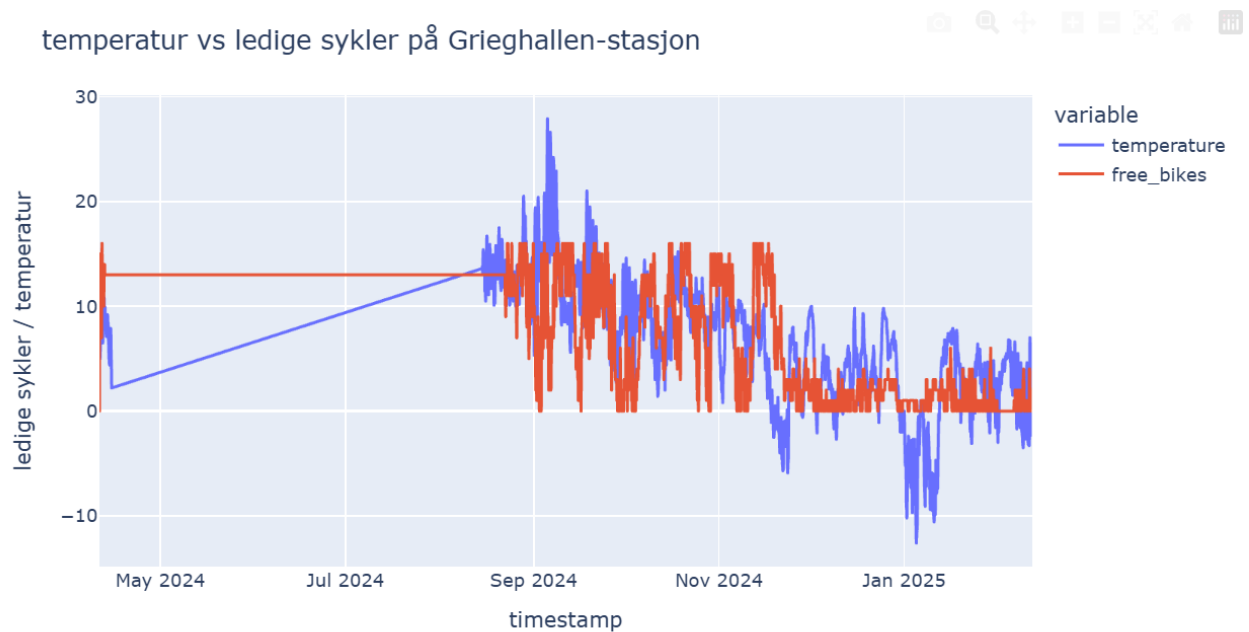
Figur 10 - Observerer sesongmønstre ved å se på vindfart i perioden



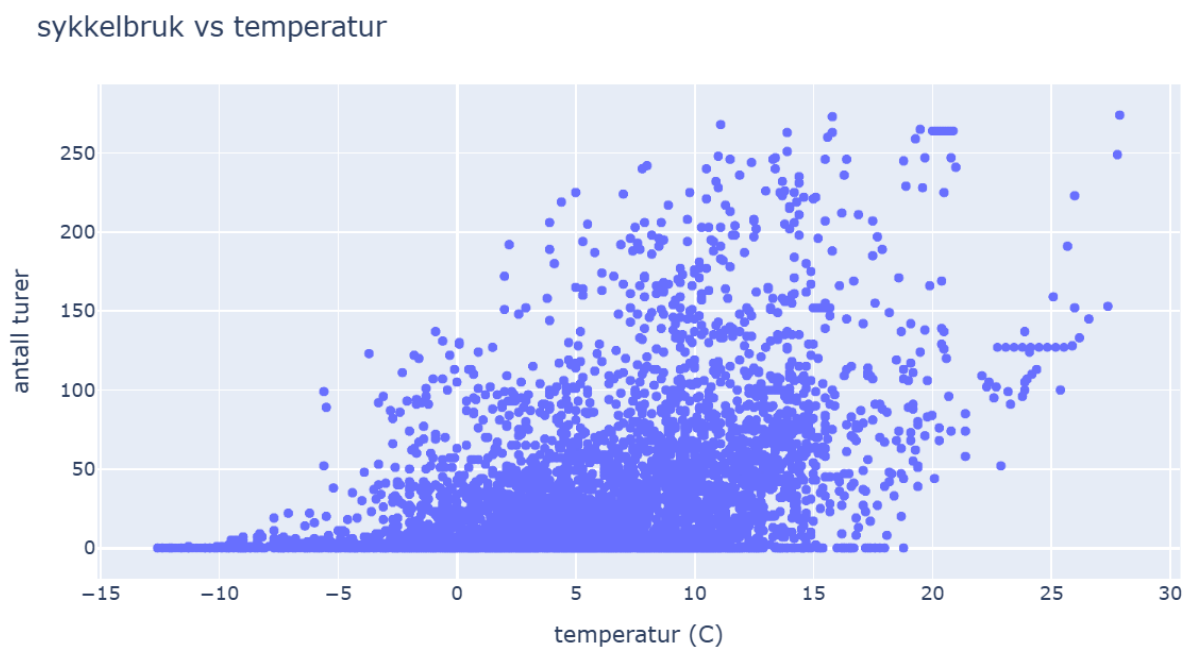
Figur 11 - Observerer sesongmønstre ved å se på nedbør i perioden*Figur 12 - Ser på sammenhengen mellom temperatur og vindfart*

merged_df:

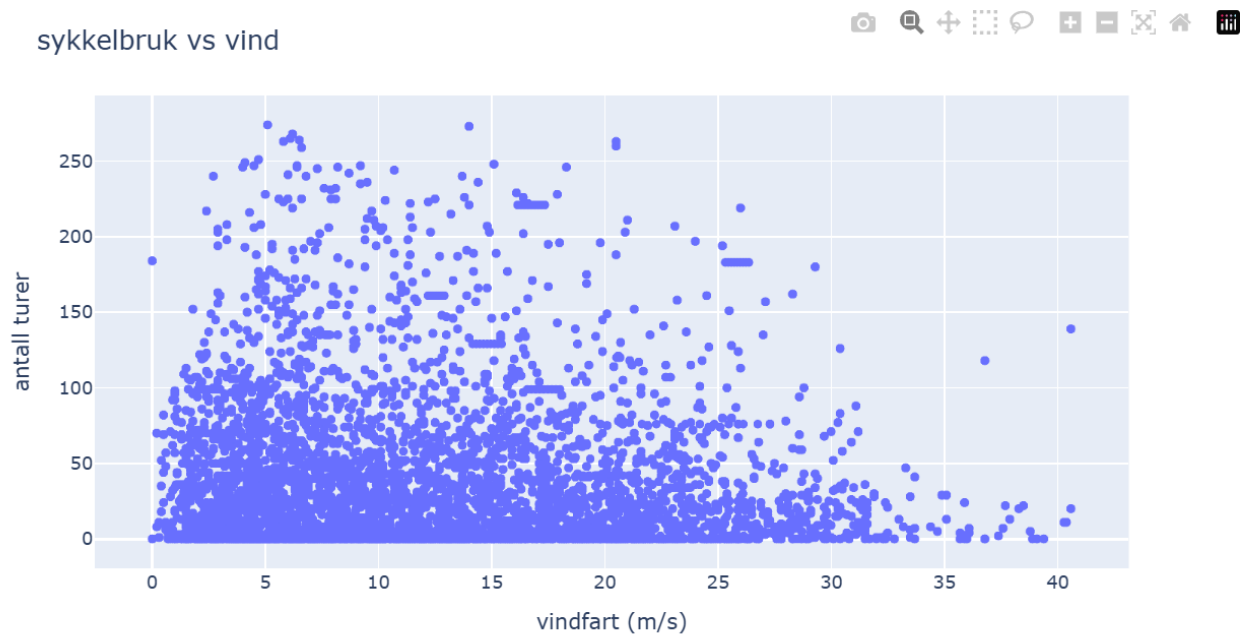
Figur 13 – Ser på korrelasjonen mellom antall ledige sykler i forhold til temperatur ved tilfeldig valgt stasjon



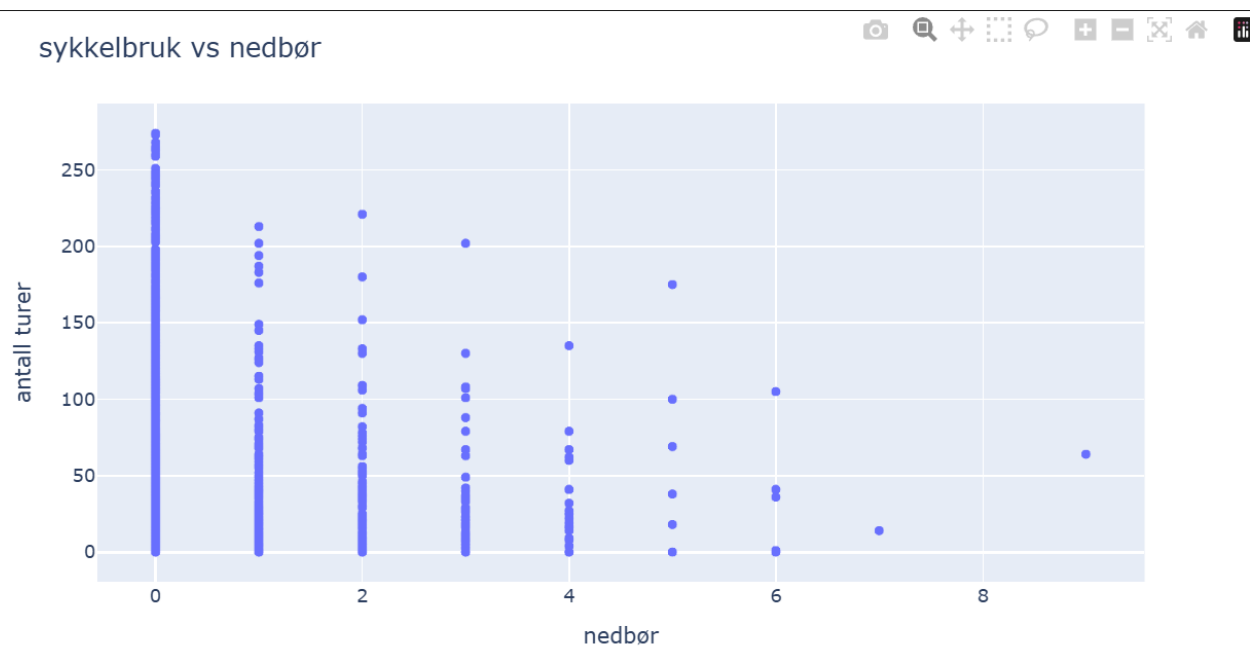
Figur 14 – Ser på korrelasjonen mellom antall sykkelturner i forhold til temperatur



Figur 15 – Ser på korrelasjonen mellom antall sykkelturner i forhold til vindfart

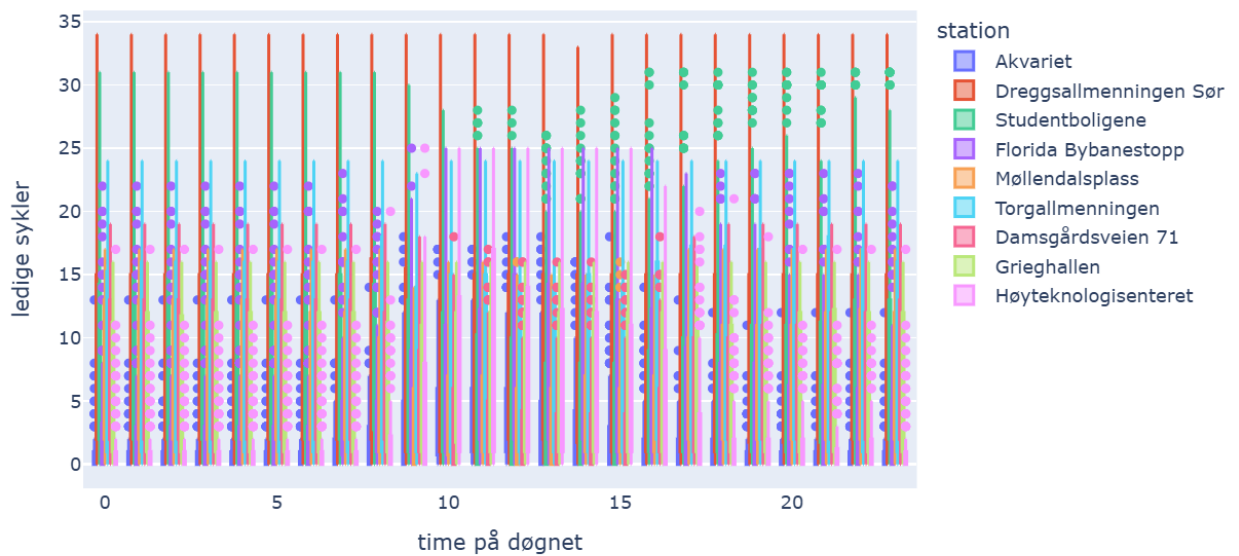


Figur 16 – Ser på korrelasjonen mellom antall sykkelturner i forhold til nedbør (observer at mange nedbørsverdier er imputert via fylling med 0)



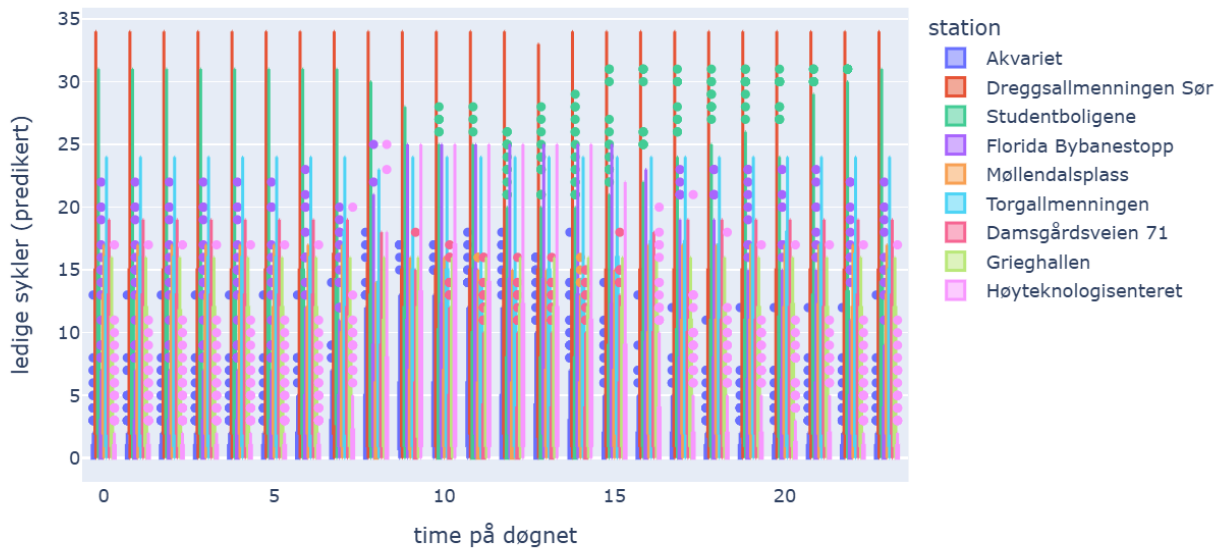
Figur 17 – Ser på ledige sykler i løpet av døgnet på mål-stasjonene

ledige sykler per time på døgnet



Figur 18 – Ser på predikert ledige sykler i løpet av døgnet på mål-stasjonene for å verifisere prediksjonslabelen

predikert ledige sykler per time på døgnet



Preprosesseringstiltak og valg

stations.csv – Endret først *timestamp* til riktig format via *pd.datetime* og konverterte til riktig tidssone. Som nevnt under dataegenskaper, så opprettet jeg et nytt datasett,

target_stations_df, som dropper alle kolonner utenom *timestamp*, *station* og *free_bikes*, men som også har et ulikt format der hver kolonne er en stasjon og hver rad er ledige sykler på et gitt tidspunkt. Dette var ønsket under visualisering, men videre brukte jeg funksjonen *pd.melt* for å omgjøre tilbake til at *free_bikes* var en kolonne og radene viste stasjonen. Omformaterte også data via LOCF, altså runde opp til neste hele time og bruke den siste registrerte verdien fram til en ny observasjon forekommer. Denne delen av preprosesseringen er svært viktig ettersom oppdateringer ikke alltid er registrert samtidig. Derfor tok jeg i bruk generativ KI for å sikre at ingen kodefeil ble gjort, og laget en visualisering for faktisk vs. antatt ledige sykler på en tilfeldig stasjon fra *target_stations_df* (se figur 2). Fjernet også «hullet» av statisk data i perioden 15.04/24 – 15.08/24, siden dette kan føre til blant annet dårlig generaliseringsevne i senere trening av prediksjonsmodeller.

trips.csv – Endret først *started_at* og *ended_at* til riktig format via *pd.datetime* og konverterte til riktig tidssone. Opprettet tre nye datasett fra rådata, der jeg sorterte ut antall turer fra en stasjon, antall ankomster og antall avganger på et gitt tidspunkt. Brukte da disse tre datasettene i sammensmeltingen.

weather.csv – Endret *timestamp* til riktig format via *pd.datetime* og konverterte til riktig tidssone, men gjorde ingen andre preprosesseringstiltak for *weather.csv* før sammensmelting.

merged_df – Dataen i *merged_df* ble sammensmeltet på *stations.csv* og all analyse ble gjort på det som ville tilsvart treningsdata (første 70%). Selve *merged_df* inneholder utvalgte kolonner fra de andre datasettene. Endret rådataen til riktig format og tidssone for *timestamp*, gjorde samme prosedyre for å hente ut *target_stations_df* som nevnt over og sorterte ut antall ankomster, avganger og total mengde turer fra *trips.csv*. Smeltet så sammen *target_stations_df*, utvalgt data fra *trips.csv* og data fra *weather.csv* via funksjonene *pd.merge_asof* og *merge*. Utledet så kolonner for time, ukedag og måned fra *timestamp* og opprettet prediksjonslabelen *free_bikes_next_hour* ved å bruke funksjonen *shift(-1)* på *free_bikes*. Sikret på denne måten at modeller som trenes på data kun får historiske og samtidige observasjoner, ikke fremtidsdata. Bruker også imputering i form av fylling av tomme rader med 0 og interpolering for å dekke manglende- og NaN-verdier der nødvendig. Omsorterte deretter kolonnene og fjernet statiske data fra

stations.csv. Ferdig merged_df ble seende slik ut:

	timestamp	hour	day_of_week	month	station	free_bikes	free_bikes_next_hour	arrivals	departures	trips_count	temperature	precipitation	wind_speed
0	2024-04-10 23:00:00+02:00	23	Wednesday	4	Akvaret	0	0.0	3	3	14	5.1	0	17.4
1	2024-04-10 23:00:00+02:00	23	Wednesday	4	Dreggsallmenningen Sør	0	12.0	1	1	14	5.1	0	17.4
2	2024-04-10 23:00:00+02:00	23	Wednesday	4	Studentboligene	0	15.0	1	1	14	5.1	0	17.4
3	2024-04-10 23:00:00+02:00	23	Wednesday	4	Florida Bybanestopp	0	0.0	1	1	14	5.1	0	17.4
4	2024-04-10 23:00:00+02:00	23	Wednesday	4	Møllendalsplass	0	4.0	1	1	14	5.1	0	17.4

Refleksjon – del 1

I del 1 av prosjektet har jeg systematisk utforsket og analysert dataen i de ulike csv-filene, og har opprettet en ryddig, fungerende og reproducerbar pipeline som tar data fra csv-filene og omformaterer til en *model_ready.csv*. Et av de viktigste stegene å merke seg er fjerningen av data der kolonnen *free_bikes* hadde en statisk verdi i en periode på omtrent 4 måneder av ukjent grunn. Dette ble gjort på anbefaling av foreleser, og sikrer bedre prediksjon tross at vi mister en del data. Jobben gjort i del 1 sikrer altså en *model_ready.csv* som er klar for modelltrening.

Del 2

Håndtering og oppsplitting av data

Håndtering av features og manglende verdier ble en enkel jobb siden alt forarbeidet ble gjort i del 1 av oppgaven. For håndtering av features, så fjernet jeg unødvendige kolonner fra alle datasettene før sammensmelting, slik at ferdig *model_ready.csv* kun inneholdt relevant informasjon. Utledning av nye features ble kun gjort direkte fra rådata, som for utledning av *timestamp*. For manglende- og NaN-verdier så fjernet jeg statiske data fra *free_bikes* i *stations.csv*, og brukte fylling av 0 og interpolering der nødvendig.

Oppsplittingen av data ble gjort ved å først opprette to datasett, X og y. X tar hele datasettet, men dropper prediksjonslabelen og timestamp (siden vi allerede har info om tid i form av time, ukedag og måned), og oppretter dummy-variabler til stasjonene for å sikre numeriske verdier i modelltreningen. Lagret senere alle kolonnene i samme mappe som den ferdige modellen for å kunne hente frem ferdige prediksjoner i de ulike stasjonene. y er prediksjonslabelen, *free_bikes_next_hour*, altså det vi ønsker å predikere. Videre brukte jeg *train_test_split*-funksjonen fra *sklearn* uten tilfeldig stokking og med satt *random-seed* til å dele opp i trenings-, validerings- og testdata. Jeg brukte første 70% av data til trening, neste 15% til validering og siste 15% til testing.

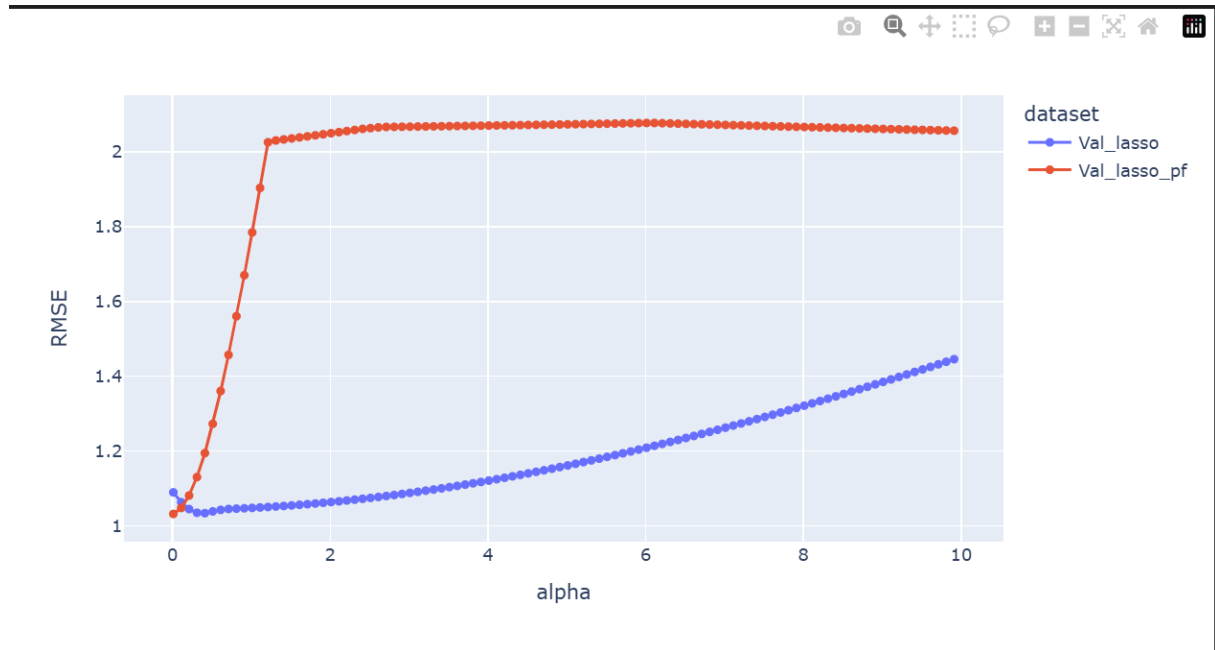
Modellvalg

Siden datasettet består av kontinuerlige variabler, så valgte jeg å trene en rekke regresjonsmodeller. Metoden for trening var stort sett lik for alle modeller, og ble gjort via *fit*-funksjonen. Ytelse ble målt via *root-mean-squared-error*-funksjonen i *sklearn* (RMSE). Jeg tok RMSE på både trenings- og valideringsdata for alle modellene, men brukte kun validerings-RMSE til å velge en ferdig modell.

Av modellene som ikke krevde spesielle hyperparametre, trente jeg modeller via *DummyRegressor*, *LinearRegression*, *Elastic Net*, *RandomForestRegressor*, *SVR* og *Ridge*. Av modellene som krevde tuning av hyperparametre, trente jeg en standard-

og en polynom-transformert Lasso-modell med verdier av alpha mellom 0 og 10 med steg på 0.1, og en KNeighborsRegressor med ulike verdier for k i intervallet [1, 5, 10, 20, 30]. Jeg visualiserte også hvordan de ulike verdiene for alpha påvirket RMSE på valideringsdata:

Figur 19 - Ser at en beste RMSE skjer på $\alpha=0.41$ for standard lasso og $\alpha=0.01$ for polynomtransformert lasso



Etter trening av modellene endte jeg opp med denne tabellen (noen av modellene mangler, siden flere med enkle hyperparametre ble satt opp i samme dictionary og kun beste ble med videre):

	model	train_rmse	val_rmse
0	DummyRegressor()	7.100	5.059
1	LinearRegression()	1.380	1.094
2	Lasso(alpha=np.float64(0.41000000000000003))	1.486	1.033
3	Lasso(alpha=np.float64(0.01))	1.283	1.031
4	KNeighborsRegressor()	1.384	1.305
5	Support Vector	1.509	1.036

Modellen med best RMSE på valideringsdata er:
Poly + Lasso, med generaliseringsevne: 1.784

Vi ser at polynom-transformert lasso (index 3, $\alpha = 0.01$) gir lavest RMSE på valideringsdata. Selve ytelsen på valideringsdata er omtrent 1.031 og generaliseringsevnen til modellen, målt på testdata, er omtrent 1.784. Dette betyr i praksis at modellen feilberegner med 1.8 sykler i gjennomsnitt i sine prediksjoner. I

forhold til baseline (index 0, `DummyRegressor()`) er dette en forskjell på over 4 feilpredikerte sykler.

For å unngå unødvendig lang kjøretid, så tok jeg kun med de 3 beste modellene + en baseline fra utforskningen videre til src-filen, *train_model.py*. La også til funksjoner for innlasting og oppsplitting av data her. En kjøring av *pipeline.py* vil nå bruke funksjonene fra *data_utils.py* og *train_models.py* til å laste inn, preprosessere og sammensmelte rådata, for så å splitte opp data og finne beste modell fra en standard og polynomtransformert Lasso-modell, en Support-Vector-Regression-modell (SVR) og en baseline `DummyRegressor()` på valideringsdata. Deretter vil den lagre den beste modellen til *best_model.pkl* via pickle, og printe RMSE på all data. Med ubegrenset tid er det nok mulig å finne en modell med lavere RMSE ved å teste enda flere imputasjonsstrategier og hyperparametre, men med tanke på kjøretid og ytelse gir disse modellene gode resultat.

Refleksjon – del 2

I del 2 av prosjektet har jeg trent en rekke regresjonsmodeller på treningsdata og målt ytelse på trenings- og valideringsdata via RMSE. Modellene jeg valgte å videreføre ble en standard og en polynomtransformert Lasso-modell, med skalering for hyperparameteren α mellom 0 og 10 med steg på 0.1, en SVR-modell og en baseline `DummyRegressor()` å sammenligne mot. Dette valget ble tatt på grunnlag av ytelse på valideringsdata. Lagde så en .py-fil, *train_model.py*, som har funksjoner for å laste inn *model_ready.csv*, splitte opp i trenings-, validerings- og testdata og trene ulike regresjonsmodeller. La også til kode i pipelinen for å lagre modellen til *best_model.pkl* og printe ytelse på all data. Har nå en ferdig trent og lagret regresjonsmodell med generaliseringsevne på omtrent 1.784 feilpredikerte sykler.

Del 3

Oppretting av filer for sammenstilling og distribusjon

Målet med del 3 var å overføre og forenkle pipeline fra del 1 og 2 til en .py-fil, *train.py*, som kjører hele pipeline fra rådata til lagret modell, og opprette en ny fil, *predict.py*, som bruker den lagrede modellen for å predikere antall ledige sykler for neste hele klokke time fra siste tidsstempel i rådata.

For *train.py* overførte jeg all kode fra tidligere pipeline og la til noen ekstra kommentarer og utskrifter for enklere visuell sjekk ved kjøring. Den ferdige filen bruker de definerte funksjonene fra *data_utils.py* og *train_model.py*. En kjøring av *train.py* vil lagre en ferdig prosessert og sammensmeltet *model_ready.csv* til output-mappen, og pkl-filer av den beste regresjonsmodellen, *best_model.pkl*, og features, *feature_cols.pkl*, til models-mappen. I tillegg vil den printe ytelsen på all data i form av RMSE:

```
-----
Lagring av csv:
-----
Laster inn data...
Preprosesserer data...
Lager features...
Lagrer csv-fil...

Ferdig! Lagret til prosjekt/output/model_ready.csv

-----
Lagring av ML:
-----
Laster inn data...
Deler opp data...
Finner beste modell og tilhørende RMSE...

Ferdig! Lagret modell til prosjekt/models/best_model.pkl
og features til prosjekt/models/feature_cols.pkl

-----
Validering:
-----
Modelltype: <class 'sklearn.linear_model._coordinate_descent.Lasso'>

Treningsdata-RMSE-en er: 1.283
Valideringsdata-RMSE-en er: 1.031
Testdata-RMSE-en er: 1.784
```

For *predict.py* brukte jeg de samme funksjonene fra *data_utils.py* til å laste inn, preprosessere og smelte sammen rådata. Brukte da rådata fra *stations.csv* til å finne siste tidsstempel i data, rundet til neste hele klokke time og brukte den til å finne prediksjonstidsstempelet. Lastet inn regresjonsmodellen og features fra kjøring av *train.py* og brukte disse til å finne predikert antall sykler på de ulike stasjonene, lagret de i en array og printet resultatene:

```
Siste tidsstempel i data: 2025-05-02 17:49:25+02:00
Neste hele klokke time: 2025-05-02 18:00:00+02:00
Predikerer for tidsstempel: 2025-05-02 19:00:00+02:00
```

Stasjon	Nåværende sykler	Predikerte sykler
Møllendalsplass	5	7
Torgallmenningen	15	15
Grieghallen	15	15
Høyteknologisenteret	23	17
Studentboligene	13	11
Akvariet	0	1
Damsgårdsveien 71	13	15
Dreggsallmenningen Sør	21	21
Florida Bybanestopp	19	15

Refleksjon – del 3

I del 3 av prosjektet har jeg overført, forenklet og utvidet pipelinen fra de tidligere delene til en fil, *train.py*, som kjører hele pipelinen fra rådata til lagret modell og printer ytelse via RMSE på all data. Har også lagd en ny fil, *predict.py*, som bruker den lagrede modellen fra kjøring av *train.py* til å lage prediksjoner for neste hele klokke time fra siste tidsstempel i rådata. Får nå printet ut ferdige prediksjoner og har fullført prosjektet i sin helhet. Oppgaven har til tider vært utfordrende, men har på veien lært svært mye om data-science-prosessen. Sluttresultatet er et produkt av et nøye arbeid med mye god konstruktiv kritikk fra fagfelle vurderinger underveis, og er absolutt noe jeg kan si meg fornøyd med.